How to generate Reports with Python (3 Formats/4 Tools)

As Excel, HTML, PDF

Lianne & Justin

July 23, 2021

Source: Unsplash

In this tutorial, we'll show you how to **generate reports with Python**.

Reporting is one of the essential tasks for anyone who works with data information. It is critical but also tedious. To free our hands and minds, we can make a program to automate the report generation process. Besides data analysis, Python is also convenient for automating routine tasks such as reporting.

Following this guide, you'll use tools with Python to generate reports, as the below common formats:

- HTML to PDF
- PDF directly

If you are looking to generate reports automatically with Python, this tutorial is a great starting point. You'll know where to start generating your next report!

We are going to show you popular and easy-to-use Python tools, with examples. The example report will include *data table*s and a *chart*, the two most common elements within reports.

Let's get started!

. . .

To follow this tutorial, you'll need to know:

- **Python basics**, which you can learn with our FREE Python crash course: breaking into Data Science.
- **Python** `pandas` **basics**, which you can learn with our course Python for Data Analysis with projects.
- **HTML basics**, which you can get a quick overview with HTML Introduction from W3 Schools.

. . .

Table Of Contents

Overview of Python reporting

Before we start, let's look at an overview of reporting with Python.

The standard formats of reports are Excel, HTML, and PDF. The good news is, Python can generate reports in all these formats. So you can choose any of these formats, depending on the needs of the report's users.

Below is a summary of what we'll cover in this tutorial. We'll need `pandas` for all the reports, since we need to manipulate and analyze data when building reports.

To generate the report as	Tools/Libraries	Use case examples
Excel	`pandas`	*if you want to use the report for further analysis in Excel*

To generate the report as	Tools/Libraries	Use case examples
HTML with template	`pandas` & `Jinja2`	*Same as above* *but for more complicated, repetitive reports, it's better to use a template*
HTML to PDF	`pandas` & `WeasyPrint`	*if you need both HTML and PDF formats* *if you are good with HTML, but need PDF as the final format*
PDF directly	`pandas` & `FPDF`	*if you only want PDF documents*

Create the example report

To show examples of the above, we'll use stock market data. We'll generate simple reports based on the historical data of the S&P 500 index from Yahoo! finance.

Below is the code to pull the data for the reports. In summary, we grab the historic data of the S&P 500 and make a summary statistics table based on it.

```
# Import libraries
# yfinance offers a reliable, threaded, and Pythonic way to download historical market data from Yahoo
# Please check out its official doc for details: https://pypi.org/project/yfinance/
import yfinance as yf
import pandas as pd

# Load historical data in the past 10 years
sp500 = yf.Ticker("^GSPC")
end_date = pd.Timestamp.today()
start_date = end_date - pd.Timedelta(days=10*365)
```

```
# Create a new column as Close 200 days moving average
sp500_history['Close_200ma'] = sp500_history['Close'].rolling(200).mean()

# Create a summary statistics table
sp500_history_summary = sp500_history.describe()
```

load_data_yfinance.py hosted with ❤ by GitHub view raw

At the end, you should have two `pandas` DataFrames `sp500_history` and `sp500_history_summary` for reporting:

- `sp500_history`: the 10 years historical data of S&P 500

  Your result will look different than below, since the date changes according to the date when you run the code.

Date	Open	High	Low	Close	Volume	Close_200
2011-07-01	1320.640015	1341.010010	1318.180054	1339.670044	3796930000	NaN
2011-07-05	1339.589966	1340.890015	1334.300049	1337.880005	3722320000	NaN
2011-07-06	1337.560059	1340.939941	1330.920044	1339.219971	3564190000	NaN
2011-07-07	1339.619995	1356.479980	1339.619995	1353.219971	4069530000	NaN
2011-07-08	1352.390015	1352.390015	1333.709961	1343.800049	3594360000	NaN

Date	Open	High	Low	Close	Volume	Close_200
2021-06-22	4224.609863	4255.839844	4217.270020	4246.439941	3208760000	3806.545(
2021-06-23	4249.270020	4256.600098	4241.430176	4241.839844	3172440000	3810.6194
2021-06-24	4256.970215	4271.279785	4256.970215	4266.490234	3141680000	3815.2927
2021-06-25	4274.450195	4286.120117	4271.160156	4280.700195	6248390000	3819.7014
2021-06-28	4284.899902	4288.410156	4276.040039	4276.399902	226596701	3824.3874

2514 rows × 6 columns

- `sp500_history_summary`: a simple summary statistics of the above data

	Open	High	Low	Close	Volume	Close_2
count	2514.000000	2514.000000	2514.000000	2514.000000	2.514000e+03	2315.00
mean	2303.293913	2314.909113	2290.775598	2303.791329	3.784200e+09	2276.84
std	719.274017	722.527344	715.816729	719.414097	9.192366e+08	609.384

	Open	High	Low	Close	Volume	Close_2
50%	2122.170044	2128.699951	2112.830078	2123.839966	3.602310e+09	2096.22
75%	2795.917480	2807.900024	2779.942444	2797.377563	4.099572e+09	2763.53
max	4284.899902	4288.410156	4276.040039	4280.700195	9.878040e+09	3824.38

Besides these two DataFrames, we'll also create a line chart showing the series of `Close` and `Close_200ma`.

The code below builds the line chart using the `matplotlib` and `seaborn` libraries, and saves it as a PNG file on your local computer. This file `chart.png` will be used in the reports below.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 sns.relplot(data=sp500_history[['Close', 'Close_200ma']], kind='line', height=3, aspect=2.0)
4 plt.savefig('chart.png')
```

seaborn_matplotlib_savefig.py hosted with ❤ by GitHub view raw

chart.png

Now we have everything needed (`sp500_history`, `sp500_history_summary`, `chart.png`) to generate reports.

To learn about using Python for data analysis, please check out our course Python for Data Analysis with projects.

The course covers the `pandas`, `seaborn` libraries. You'll learn how to manipulate data, create data visualizations, etc., which is essential to create reports in Python.

Let's dive into each specific example!

Excel

We'll start with the classic: Excel. Excel is a widely used, powerful data analysis and visualization tool. In many scenarios, you may want to store the report within Excel and share it with others. Others can easily open the spreadsheet, examine the report, and even use it for further analysis in Excel, Python, or other programs.

While there are different tools to save reports as Excel, we'll use the foundation one: `ExcelWriter` and the `to_excel` method in `pandas`. Here is the summary of the steps. You can read it together with the code below.

1. **Set up an `ExcelWriter`** with engine 'openpyxl'

   You can also use the engine 'xlsxwriter'. The general procedure is the same, but the syntax will be different. We are using 'openpyxl' since it's the default engine for xlsx files.
2. **Export the data** from Python **to Excel**

   We export DataFrames `sp500_history` and `sp500_history_summary` to two separate sheets/tabs.
3. **Add a line chart in Excel** showing the data of `Close` and `Close_200ma`

```python
# Import libraries
from openpyxl.chart import LineChart, Reference

# 1. Set up an ExcelWriter
with pd.ExcelWriter('excel_report.xlsx', engine='openpyxl') as writer:
# 2. Export data
    sp500_history.to_excel(writer, sheet_name='historical_data')
    sp500_history_summary.to_excel(writer, sheet_name='historical_data_summary')

# 3. Add a line chart
    # Point to the sheet 'historical_data', where the chart will be added
    wb = writer.book
    ws = wb['historical_data']
    # Grab the maximum row number in the sheet
    max_row = ws.max_row
    # Refer to the data of close and close_200ma by the range of rows and cols on the sheet
    values_close = Reference(ws, min_col=5, min_row=1, max_col=5, max_row=max_row)
    values_close_ma = Reference(ws, min_col=7, min_row=1, max_col=7, max_row=max_row)
    # Refer to the date
    dates = Reference(ws, min_col=1, min_row=2, max_col=1, max_row=max_row)
    # Create a LineChart
    chart = LineChart()
    # Add data of close and close_ma to the chart
    chart.add_data(values_close, titles_from_data=True)
    chart.add_data(values_close_ma, titles_from_data=True)
    # Set the dates as the x axis and format it
    chart.set_categories(dates)
    chart.x_axis.number_format = 'mmm-yy'
    chart.x_axis.majorTimeUnit = 'days'
    chart.x_axis.title = 'Date'
    # Add title to the chart
    chart.title = 'Close prices of S&P 500'
    # Refer to close_ma data, which is with index 1 within the chart, and style it
    s1 = chart.series[1]
    s1.graphicalProperties.line.dashStyle = 'sysDot'
    # Add the chart to the cell of G12 on the sheet ws
    ws.add_chart(chart, 'G12')
```

There is much more styling you can accomplish with this method. Please check out `openpyxl` documentation.

HTML

HTML (Hyper Text Markup Language) is the standard markup language for creating web pages. We can embed an HTML format report easily on a web page, or an email. So it is also popular for different use cases.

We'll cover two main methods of generating HTML reports in Python. One is the basic one, and the other is to generate one with templates using the library called `Jinja2`.

Let's start with the basic one. We can define HTML code as a Python string, and write/save it as an HTML file.

Here are the general steps of the procedure. You can read it together with the code below.

1. **Set up multiple variables** to store the titles, text within the report
2. **Combine them as a long f-string** of code

   Within the f-string, we define the HTML document, including:

   - the head: contains meta information about the HTML page, including the title
   - the body: a container for all the visible contents, such as `h1`, `p`, `img`, `table`

     Within the `img` element, we include the `chart.png` file saved on your computer.

     To include the DataFrame as HTML `tables`, we use the `to_html` method. For simplicity, we only render the last 3 rows of the DataFrame `sp500_history`.
3. **Write the `html` string as an HTML file**

```python
stats_text = 'Historical prices summary statistics'



# 2. Combine them together using a long f-string
html = f'''
    <html>
        <head>
            <title>{page_title_text}</title>
        </head>
        <body>
            <h1>{title_text}</h1>
            <p>{text}</p>
            <img src='chart.png' width="700">
            <h2>{prices_text}</h2>
            {sp500_history.tail(3).to_html()}
            <h2>{stats_text}</h2>
            {sp500_history_summary.to_html()}
        </body>
    </html>
    '''
# 3. Write the html string as an HTML file
with open('html_report.html', 'w') as f:
    f.write(html)
```

python_report_html.py hosted with ❤ by GitHub view raw

You can open the final HTML report in any modern browser. You can try to match each of the HTML elements to the final report below.

We'll leave the report as it is. If you want to style this HTML report more, please learn about CSS (Cascading Style Sheets). With CSS, you can control almost everything, including the color of text, font, spacing between elements, background color, and so on.

HTML with template

So we can easily create an HTML template, then use it with Python-like syntax. This is especially useful when you are using complicated templates for reporting over and over again.

Below are the general steps to use `Jinja2` to generate HTML reports:

1. **Create an HTML template with variables**

   I named it 'report_template.html', and saved it under a folder in the current working directory called 'templates'.

Within such a file, I put in the code below.

We are creating the same HTML report as the previous example, but the syntax of `Jinja`'s template is different from Python strings. You can read more about its syntax here.

```
<html>
    <head>
        <title>{{page_title_text}}</title>
    </head>
    <body>
            <h1>{{title_text}}</h1>
            <p>{{text}}</p>
            <img src='chart.png' width="700">
            <h2>{{prices_text}}</h2>
            {{sp500_history.tail(3).to_html()}}
            <h2>{{stats_text}}</h2>
            {{sp500_history_summary.to_html()}}
    </body>
</html>
```

html_template_jinja2_file.py hosted with ❤️ by GitHub view raw

2. **Create a template** `Environment` object, which will be used to load templates

   `loader=FileSystemLoader('templates')` tells `Jinja` to look for templates stored within a file/folder called 'templates'. You can use other `loader`s to load templates in other ways or from other locations, please check the doc for details.

3. **Load the template from the** `Environment`

   We load the template file set up earlier called 'report_template.html'.

4. **Render the template with variables**

   As you can see, the variables within `{{...}}` in 'report_template.html' are set as specific values. `Jinja` will embed them within the template.

5. **Write the template to an HTML file**

   You can open the HTML file in any modern web browser. It should look the same as the previous example of HTML report.

```
from jinja2 import Environment, FileSystemLoader

```

```
8
9  # 4. Render the template with variables
10 html = template.render(page_title_text='My report',
11                        title_text='Daily S&P 500 prices report',
12                        text ='Hello, welcome to your report!',
13                        prices_text='Historical prices of S&P 500',
14                        stats_text='Historical prices summary statistics',
15                        sp500_history=sp500_history,
16                        sp500_history_summary=sp500_history_summary)
17
18 # 5. Write the template to an HTML file
19 with open('html_report_jinja.html', 'w') as f:
20     f.write(html)
```

html_template_jinja2.py hosted with ❤ by GitHub view raw

You might wonder, `Jinja2` doesn't seem to be very impressive in this example? It seems similar to the basic approach, except that we use some `Jinja2` methods versus the f-string. But imagine if you have a much more complicated report, and you want to reuse it, then `Jinja` would make it much easier.

Using the below code, we convert the HTML file 'html_report_jinja.html' to a PDF file called 'weasyprint_pdf_report.pdf', with an inline CSS stylesheet. Within the stylesheet, we specified the page size, margin, and the table header and cell border.

```
from weasyprint import HTML, CSS
css = CSS(string='''
    @page {size: A4; margin: 1cm;}
    th, td {border: 1px solid black;}
    ''')
HTML('html_report_jinja.html').write_pdf('weasyprint_pdf_report.pdf', stylesheets=[css])
```

html_to_pdf_weasyprint.py hosted with ❤ by GitHub view raw

The final result is a nice looking PDF below.

PDF directly

What if you're not familiar with HTML, you just want the PDF format report? Python also has a solution for that. We'll use this Python package called `FPDF`.

The challenge to generate our report with FPDF is to show the tables of data. So before using FPDF, we define a function below to loop through the columns and rows of a DataFrame to display it on cells in the PDF.

```python
def output_df_to_pdf(pdf, df):
    # A cell is a rectangular area, possibly framed, which contains some text
    # Set the width and height of cell
    table_cell_width = 25
    table_cell_height = 6
    # Select a font as Arial, bold, 8
    pdf.set_font('Arial', 'B', 8)

    # Loop over to print column names
    cols = df.columns
    for col in cols:
        pdf.cell(table_cell_width, table_cell_height, col, align='C', border=1)
    # Line break
    pdf.ln(table_cell_height)
    # Select a font as Arial, regular, 10
    pdf.set_font('Arial', '', 10)
    # Loop over to print each data in the table
    for row in df.itertuples():
        for col in cols:
            value = str(getattr(row, col))
            pdf.cell(table_cell_width, table_cell_height, value, align='C', border=1)
        pdf.ln(table_cell_height)
```

fpdf_function_dataframe_table.py hosted with ❤ by GitHub view raw

Then, we can use FPDF to display the report. The general procedure is:

1. **Set up basic setting for a PDF page with** FPDF
2. **Lay out each item** we want to see in the report

Please see the details in the comments below. It is helpful to match the code with the

```
2
3  # 1. Set up the PDF doc basics
4  pdf = FPDF()
5  pdf.add_page()
6  pdf.set_font('Arial', 'B', 16)
7
8  # 2. Layout the PDF doc contents
9  ## Title
10 pdf.cell(40, 10, 'Daily S&P 500 prices report')
11 ## Line breaks
12 pdf.ln(20)
13 ## Image
14 pdf.image('chart.png')
15 ## Line breaks
16 pdf.ln(20)
17 ## Show table of historical data
18 ### Transform the DataFrame to include index of Date
19 sp500_history_pdf = sp500_history.reset_index()
20 ### Transform the Date column as str dtype
21 sp500_history_pdf['Date'] = sp500_history_pdf['Date'].astype(str)
22 ### Round the numeric columns to 2 decimals
23 numeric_cols = sp500_history_pdf.select_dtypes(include='number').columns
24 sp500_history_pdf[numeric_cols] = sp500_history_pdf[numeric_cols].round(2)
25 ### Use the function defined earlier to print the DataFrame as a table on the PDF
26 output_df_to_pdf(pdf, sp500_history_pdf.tail(3))
27 ## Line breaks
28 pdf.ln(20)
29 ## Show table of historical summary data
30 sp500_history_summary_pdf = sp500_history_summary.reset_index()
31 numeric_cols = sp500_history_summary_pdf.select_dtypes(include='number').columns
32 sp500_history_summary_pdf[numeric_cols] = sp500_history_summary_pdf[numeric_cols].round(2)
33
34 output_df_to_pdf(pdf, sp500_history_summary_pdf)
35 # 3. Output the PDF file
36 pdf.output('fpdf_pdf_report.pdf', 'F')
```

fpdf_report.py hosted with ❤ by GitHub view raw

The final PDF report looks like below

You might have heard about another popular tool called `ReportLab`. We picked `FPDF` since it seems to be easier to learn.

. . .

In summary, you've learned examples of using Python to generate reports as Excel, HTML

- If you want an Excel file, then do Excel
- If you want to embed HTML to web pages, or are just good at HTML, use HTML. You can also convert HTML to PDF after
- If you only want PDF, you can go with PDF directly too

Each of these methods/packages has a lot more techniques. Hope you got the basics within this tutorial, and are ready to explore details on your own!

After generating reports using Python, it is also convenient to automatically send emails to share the reports. Please check out How to Send Emails using Python: Tutorial with examples.

> **Further learning**: if you want an interactive web-based report/dashboard, we highly recommend `plotly Dash`.
> You can either take our comprehensive introductory course: Python Interactive Dashboards with Plotly Dash, or read our article with an example: 6 Steps to Interactive Python Dashboards with Plotly Dash.

We'd love to hear from you. Leave a comment for any questions you may have or anything else.

AUTOMATION, DATA SCIENCE, EXCEL REPORT, HTML, PDF, PYTHON REPORT, PYTHON REPORTING, REPORT GENERATION

« Unlocking Random Forest in Machine Learning

How to Send Emails using Python: Tutorial with examples »

7 thoughts on “How to generate Reports with Python (3 Formats/4 Tools)
  As Excel, HTML, PDF ”

EDWIN CABEZAS
OCTOBER 26, 2021 AT 6:59 PM

Que buenos ejemplos.
Muchas gracias.

Reply

LIANNE & JUSTIN
OCTOBER 27, 2021 AT 6:00 PM

🙄

Reply

JOHN THE JACK
DECEMBER 2, 2021 AT 4:51 AM

Reply

LIANNE & JUSTIN
DECEMBER 3, 2021 AT 9:43 PM

Thanks for the suggestion, John

Reply

RUNY
FEBRUARY 4, 2022 AT 5:25 AM

Hi
Thanks.

Is there a way to create nice tables with nice headers and colour formatting and add dynamic data from dataframes into them and then paste those tables in Word? I meed to produce a final document in Word. I'm using the docx library now.

Is there a way to include a table of contents in your Word document and your html document from python?

Reply

LIANNE & JUSTIN
FEBRUARY 5, 2022 AT 10:52 AM

Hi Runy, we've never tried to do that in Word so sorry can't help.

Reply

Hi there, we have an open-source library for creating reports using Python that also may be helpful! You can find it at https://datapane.com / https://github.com/datapane/datapane.

If you want to jump straight in, here are the docs:
https://docs.datapane.com/reports/overview/

Reply

Leave a Comment

Your email address will not be published. Required fields are marked *

Type here..

Name*

Email*

Post Comment »

More recent articles

How to build XGBoost models in Python

With a step-by-step example

Lianne & Justin / January 16, 2023

This is a practical guide to XGBoost in Python.
Learn how to build your first XGBoost model with this step-by-step tutorial.

Read More »

What is gradient boosting in machine learning: fundamentals explained

Must read before implementing

Lianne & Justin / November 22, 2022

This is a beginner's guide to gradient boosting in machine learning.
Learn what it is and how to improve its performance with regularization.

Read More »

What are Python errors and How to fix them

Lianne & Justin / October 4, 2022

This is a tutorial to Python errors for beginners. Learn their types and how to fix them with general steps.

Read More »

This blog is *just* for you, who's into data science!
And it's created by people who are *just* into data.

We are the brains of *Just into Data*. We created this blog to share our interest in data with you.

If you are into data science as well, and want to keep in touch, sign up our email newsletter. We're on Twitter, Facebook, and Medium as well.

Newsletter

Get regular updates straight to your inbox:

First Name

Email Address

Sign Up